VT-024 | Bounded retry cho GitHub API

GitHub Project Coach · Báo cáo kiểm thử và đánh giá kỹ thuật

Thông tin	Giá trị
Họ tên / MSSV	Hoàng Khánh An / 23001819
Team / mã đề tài	SE2026-T07 / AI-03
Repository	duyendojhs/SE2026-15-1 (đề bài); repo dự án: SE2026-T07
Nhánh / PR	bt3-23001819-vt024-retry · PR #15 (draft)
Baseline	90f831c8c8afc84dd4f0268b7ed5704e41f32cbf

"Tôi chịu trách nhiệm về toàn bộ code, test và nội dung bài làm; tôi đã khai báo việc sử dụng AI."

Bối cảnh và yêu cầu cá nhân hóa

GitHub Project Coach đồng bộ issue, pull request, review và metadata vào backend. VT-024 yêu cầu GitHubClient chịu được lỗi tạm thời trong các lần đọc an toàn, trong khi giới hạn số lần thử và giữ nguyên lỗi khi cạn lượt. Điểm thay đổi được chọn là tầng HTTP client; yêu cầu không bao gồm retry toàn bộ backfill.

Vấn đề và rủi ro chính

Nếu GET bị timeout hoặc mất kết nối nhất thời, yêu cầu đọc có thể thất bại dù lần gọi tiếp theo thành công. Ngược lại, retry POST/PUT/PATCH/DELETE có thể lặp side effect. Backfill còn ghi RawEvent và AuditLog theo kiểu append; uniqueness của business rows không chứng minh các bản ghi phụ có exactly-once semantics. Vì vậy phạm vi retry phải hẹp, có giới hạn và không biến lỗi persistence thành kết quả thành công.

Kết quả cuối: thay đổi production giới hạn ở GitHubClient; kiểm thử cuối 49 passed; branch đã được đẩy lên remote.

02 / ĐẶC TẢ VÀ BẤT BIẾN

Tiêu chí	Hành vi được xác nhận
AC-01 · retry tạm thời	GET retry khi gặp 5xx, timeout hoặc ConnectError; dùng giới hạn và backoff hiện có.
AC-02 · không retry sai	401/404 không retry; request không an toàn không retry sau status/transport failure; hết lượt phải báo lỗi.
AC-03 · ingest ổn định	Re-ingestion giữ duy nhất business rows theo các khóa/upsert hiện có; không khẳng định exactly-once cho raw/audit append.
AC-04 · persistence	Lỗi commit không được trả về BackfillResult thành công; API cũng không được trả success giả.
AC-05 · partial failure	Khi một nguồn lỗi nhưng nguồn khác thành công, trạng thái partial và dữ liệu hợp lệ được giữ.

Bất biến kỹ thuật

Số lần gọi ≤ max_retries + 1; max_retries=0 nghĩa là đúng một lần thử. Kết quả thành công chỉ trả sau phản hồi thành công. Khi cạn lượt, transport exception hoặc GitHubAPIError phù hợp được truyền ra. Endpoint allowlist và auth header tiếp tục được áp dụng.

Yêu cầu phi chức năng và ràng buộc

Mục tiêu kiểm thử hiệu năng: 100 concurrent users/tasks và p95 latency ≤ 3.0 giây. Kiểm thử tự động phải offline và deterministic. Privacy policy chỉ lưu metadata, không lưu nội dung issue/comment nhạy cảm. Retry phải bounded và không tạo side effects trùng ngoài ý muốn. Bản vá retry GET cho ConnectError/TimeoutException, không thêm dependency runtime, giữ backoff hiện hữu, không retry unsafe methods và không mở rộng sang BackfillService. Benchmark mock ở trang 7 không chứng minh production đạt p95 ≤ 3.0 giây.

Định nghĩa failure rõ ràng

Lỗi kết nối/timeout sau lượt cuối không bị đổi thành success hoặc che giấu. HTTP 401/404 giữ nguyên xử lý không retry. 403/429/5xx cho GET giữ xử lý hiện hành; với non-GET, giới hạn retry không được làm thay đổi kiểu exception hiện hữu.

03 / KẾ HOẠCH AI VÀ REVIEW CỦA CON NGƯỜI

Kế hoạch triển khai ban đầu

1) Kiểm tra hợp đồng GitHubClient, allowlist, backoff và trạng thái backfill. 2) Viết test trước cho timeout, ConnectError, retry bound, unsafe POST và persistence failure. 3) Quan sát RED và giữ log nguyên gốc. 4) Chỉ sửa request loop trong backend/app/github/client.py. 5) Chạy GREEN, hồi quy, lint rồi đánh giá mutation, coverage và harness mock. 6) Ghi AI Usage Log và traceability trước khi đóng gói báo cáo.

Quyết định review 1 — không retry toàn workflow

Không bọc BackfillService.run() bằng retry. Kiểm tra models.py xác nhận uniqueness/upsert cho business rows, nhưng RawEvent và AuditLog được append trong cùng unit of work và không có khóa idempotency tương ứng. Chạy lại workflow có thể nhân bản log phụ. Tầng GET là nơi an toàn hơn cho retry trong vertical slice.

Quyết định review 2 — Hypothesis là phần bắt buộc

Property test không bị bỏ qua sau khi example-based test pass. Oracle ban đầu đòi GitHubAPIError sau transport exhaustion; kiểm tra client hiện hữu cho thấy httpx transport exception được truyền nguyên trạng và không có contract yêu cầu chuyển kiểu. Oracle được sửa sang httpx.TransportError, nhưng vẫn giữ bounded attempts và yêu cầu không trả success khi cạn lượt.

Quyết định review 3 — giữ bằng chứng RED

Log RED gốc và test-only commit được giữ nguyên. Bốn test thất bại chính xác: test_read_timeout_is_retried_then_succeeds, test_connection_error_is_retried_then_succeeds, test_transient_http_error_on_post_is_not_retried, test_timeout_retry_is_bounded_and_returns_only_after_success. Persistence characterization test_commit_failure_never_returns_success_and_recovery_is_consistent đã PASS trên baseline. API regression ban đầu kỳ vọng JSON 500; phản hồi thực tế của Starlette là plain text nên chỉ assertion được chỉnh theo response quan sát được.

Rủi ro thiếu ngữ cảnh đã điều tra

Sau khi được phép kiểm tra model/audit code, xác nhận business rows có uniqueness nhưng raw events/audit records không có cùng bảo đảm. Rủi ro còn lại: manual full backfill có thể append thêm log; chưa có test count cho side effect này. Nhật ký AI ghi lại giả định, quyết định review, và giới hạn.

04 / CHIẾN LƯỢC KIỂM THỬ THEO RỦI RO

T01–T14 là documentation IDs của báo cáo, không đổi tên Python test function hiện có.

ID	Requirement / Risk	Test Level	Precondition / Input	Expected Result	Priority
T01 test_rate_limit_retry_after_is_respected	AC-01 Retry-After	Unit	GET 403 + Retry-After	Sleep delay follows header	P2
T02 test_rate_limit_exhaustion_raises	AC-02 Bound/exhaustion	Unit	GET receives repeated 403	Raise GitHubAPIError at bound	P1
T03 test_bad_token_is_not_retried	AC-02 Auth	Unit	GET returns 401	One attempt; propagate error	P1
T04 test_content_endpoint_is_blocked_before_network	Allowlist	Unit	Endpoint outside allowlist	Reject before network	P1
T05 test_pagination_follows_link_header	Pagination	Unit	Response includes Link header	Follow next page	P2
T06 test_http_500_is_retried_then_succeeds	AC-01 Server error	Unit	GET 500 then 200	Retry and return success	P1
T07 test_read_timeout_is_retried_then_succeeds	AC-01 Timeout	Unit	GET ReadTimeout then 200	Retry and return success	P1
T08 test_connection_error_is_retried_then_succeeds	AC-01 ConnectError	Unit	GET ConnectError then 200	Retry and return success	P1
T09 test_transient_http_error_on_post_is_not_retried	AC-02 Unsafe method	Unit	POST receives 500	One request; propagate error	P1
T10 test_timeout_retry_is_bounded_and_returns_only_after_success	AC-02 Retry bound	Property	max_retries 0–5; transient sequence	≤ retries+1; exhaustion raises	P1
T11 test_backfill_is_idempotent_and_metadata_only	AC-03 Re-ingestion	Integration	Repeat same backfill	Business-row counts stable	P1
T12 test_backfill_marks_partial_when_one_resource_fails	AC-05 Partial failure	Integration	One resource fails; others succeed	Partial status; keep successful data	P1
T13 test_generated_duplicate_issue_ids_remain_unique_after_reordered_reingestion	AC-03 Duplicate IDs	Property	Duplicate IDs in changed order	Issue rows remain unique	P1
T14 test_commit_failure_never_returns_success_and_recovery_is_consistent	AC-04 Persistence	Integration	Inject commit failure; then recover	No false success; rollback/recovery	P1

Mức kiểm thử và oracle

MockTransport cho HTTP/unit; SQLite in-memory cho backfill/integration; TestClient cho API persistence regression sau đó; không chạy E2E với GitHub thật. Retry property sinh max_retries 0–5 và chuỗi timeout/connection. Oracle đếm lần gọi, xác nhận ≤ max_retries+1 và yêu cầu exception khi không còn lượt. Invariant idempotency kiểm tra business rows sau ID trùng và reordering.

Hypothesis thực thi: retry 40 pass, 2 invalid, 0 fail; idempotency 30 pass, 4 invalid, 0 fail. Không có minimized failing counterexample. Nguyên nhân chi tiết của invalid examples không được log.

05 / TEST-FIRST: RED → PATCH → GREEN

Mốc	Commit	Bằng chứng
Baseline	90f831c8c8afc84dd4f0268b7ed5704e41f32cbf	Điểm bắt đầu; client chưa retry timeout/connection.
RED · test-only	11055810f10729963a8706b22fec2fcf5d9b0647	4 test fail: ReadTimeout, ConnectError, POST 500 retry, bounded retry. Persistence characterization đã pass baseline.
PATCH · GREEN	ecd22676f6a1ce43bcfa0d21852f717d475c130d	Production chỉ backend/app/github/client.py; oracle dùng TransportError.
Quality evidence	e4e56c86b9dab3e3c2fbaf36b751f29e4b135a	Mutation, coverage, API regression, lint/perf evidence.
Perf refinement	5f1fcbb719a8efad847e58bca03dce406566bfae	Barrier 100 workers; latency tính cả outcome lỗi.
Traceability	4982995c17d42999908780b3aec1af6aa4c7cd05	Ma trận test/AC, tên test thật và SHA.
Final verification	99ca2d46841be8a164224c9ac7597961fa61b58f	49 passed; Ruff pass; clean branch trước hồ sơ.

Bản vá tối thiểu

Request loop retry ConnectError và TimeoutException cho GET nếu còn lượt; chờ theo exponential backoff hiện hữu. Hết lượt thì raise exception gốc. Các response 403/429/5xx tiếp tục theo logic cũ; non-GET không nhận retry status/transport. Không thay đổi BackfillService, allowlist, token provider hay dependency production.

GREEN và hồi quy

Checkpoint 3: 14/14 test retry/backfill. Checkpoint 4: 48/48 toàn backend. Sau API persistence regression: 49/49 cuối cùng (một warning deprecation). Ruff: All checks passed. Thứ tự ba file test được đảo hai lần, mỗi lần 15/15 pass. Mypy không chạy được vì module không tồn tại trong .venv.

Vì sao GitHubClient đủ cho lát cắt

Lỗi phát sinh trước khi có dữ liệu trả về từ GitHub; retry đúng một GET trong client giữ nguyên ranh giới và phép đo số lần gọi. Backfill không cần biết transport retry, và lỗi commit vẫn được kiểm tra riêng ở service/API. Phạm vi hẹp tránh nhân đôi side effects của workflow.

06 / MUTATION VÀ COVERAGE

Mutation thử nghiệm

Tạo mutant tạm thời bắt lỗi session.commit() trong BackfillService.run() rồi trả BackfillResult giả ở trạng thái ready. Chạy test_commit_failure_never_returns_success_and_recovery_is_consistent cho kết quả DID NOT RAISE SQLAlchemyError: test thất bại đúng kỳ vọng và mutant bị giết. Diff mutant và output được giữ trong evidence. Kiểm tra sau đó: backfill.py không có diff so với baseline; không có mutation trong source branch.

Module / phạm vi	Statement	Branch	Combined Cover
app/github/client.py	95/121 · 78.51%	31/50 · 62.00%	73.68% (74%)
app/ingestion/backfill.py	151/163 · 92.64%	35/40 · 87.50%	91.63% (92%)
Tổng hai module	246/284 · 86.62%	66/90 · 73.33%	83.42% (83%)

Coverage command giới hạn source ở hai module production và chạy 15 test retry/backfill/API. Cột Combined là số coverage.py gộp statement và branch; không được gọi là line coverage. Đây không phải coverage toàn repository.

Rủi ro được bao phủ

AC-01: 5xx, timeout, ConnectError và bound có test trực tiếp. AC-02: 401, POST/500, 403 exhaustion và transport exhaustion; chưa có test riêng 404, generic 4xx hay POST transport. AC-03: business rows được kiểm tra; raw/audit count không được assert. AC-04: service, API và mutant đều kiểm tra false success. AC-05: partial một nguồn lỗi được kiểm tra; all-fail path chưa có test riêng.

Branch còn thiếu có giá trị rủi ro

Client: rate-reset wait, Retry-After thiếu/sai, generic 4xx, 304, pagination malformed. Backfill: all-resources-failed, issue cũng là PR, cutoff PR cũ, review thiếu PR. Coverage mô tả phần chưa chạy; tỷ lệ cao không thay thế assertion theo rủi ro.

07 / ĐỘ ỔN ĐỊNH VÀ HIỆU NĂNG

Điều tra rủi ro flaky đã thực thi

Time/backoff: test dùng sleep callback giả ghi nhận delay, không đợi thật; jitter fallback không bị assert. Hypothesis: bật deterministic seed behavior (derandomize=True), không dùng example database; retry 40 pass/2 invalid, ingest 30 pass/4 invalid. Test order: hai thứ tự file khác nhau đều 15 pass; SQLite in-memory theo fixture và API listener/override được dọn trong finally. Concurrency: harness có 100 worker, mỗi task sở hữu client/transport/counter riêng; không gọi GitHub thật.

Thước đo mock	Kết quả
Task / worker	100 / 100; barrier chặn trước khi đo tới khi đủ worker
Scenario	90 success ngay; 5 transient rồi success; 5 exhausted có chủ ý
Request / outcome	115 request = 90×1 + 5×2 + 5×3; 95 success, 5 fail dự kiến
Latency trên 100 task	p50 0.376 ms · p95 0.701 ms · max 1.131 ms
Đo lường	perf_counter mỗi task; p95 nearest-rank trên cả 100 outcome, kể cả lỗi

Diễn giải đúng phạm vi

100 worker rendezvous tại Barrier trước timed interval, nên tác vụ được thả đồng thời; thời gian barrier không tính latency. Handler MockTransport không blocking. Sleep/backoff là no-op: latency đo request/retry logic và callback, nhưng loại trừ thời gian chờ backoff thật. Năm lỗi exhausted được tiêm có chủ ý; 115 request khớp với cấu hình max_retries=2.

Giới hạn

Đây là benchmark trong process với mock. Nó không đánh giá p95 production, độ trễ GitHub thật, database throughput, API concurrency end-to-end hay chứng minh ngưỡng production p95 ≤ 3 giây. Không có timing assertion trong pytest. Đây chỉ là bằng chứng rằng harness hoạt động theo các chuỗi retry đã định.

08 / TRACEABILITY, BẰNG CHỨNG VÀ PHẢN TƯ

AC / rủi ro · case IDs	Test function(s)	File · commit	Kết quả
AC-01/02 · T01, T02, T06	test_rate_limit_retry_after_is_respected; test_rate_limit_exhaustion_raises; test_http_500_is_retried_then_succeeds	test_rate_limit.py · 1105581	Pass; GET retry/exhaustion
AC-02 · T03, T04, T05	test_bad_token_is_not_retried; test_content_endpoint_is_blocked_before_network; test_pagination_follows_link_header	test_rate_limit.py · 1105581	Pass; 401, allowlist, pagination
AC-01/02 · T07–T10	test_read_timeout_is_retried_then_succeeds; test_connection_error_is_retried_then_succeeds; test_transient_http_error_on_post_is_not_retried; test_timeout_retry_is_bounded_and_returns_only_after_success	test_rate_limit.py · 1105581 / ecd2267	Pass; property 40 pass, 2 invalid
AC-03/05 · T11–T13	test_backfill_is_idempotent_and_metadata_only; test_backfill_marks_partial_when_one_resource_fails; test_generated_duplicate_issue_ids_remain_unique_after_reordered_reingestion	test_backfill.py · 7c13113 / 1105581	Pass; property 30 pass, 4 invalid
AC-04 · T14	test_commit_failure_never_returns_success_and_recovery_is_consistent	test_backfill.py · 1105581	Pass; mutant killed
API regression · ngoài T01–T14	test_sync_api_does_not_return_success_when_persistence_fails	test_tenants_sync_api.py · e4e56c8	Pass; API 500
Performance experiment	main / _run_task	perf_retry_harness.py · e4e56c8 / 5f1fcbb	100 task mock; p95 0.701 ms

T01–T14 là documentation IDs, ánh xạ từng test function tại trang 4; tên Python test không đổi. API persistence regression được thêm sau đó và nằm ngoài bộ 14 case. Commit đầy đủ, commands, mutation experiment và kết quả nằm trong evidence appendix. Hai giả định AI đã sửa: ép transport exhaustion thành GitHubAPIError (oracle sửa về httpx.TransportError); kỳ vọng response 500 JSON (assert plain text thực tế). Tóm tắt perf sai 85/10/5 được sửa thành 90/5/5 theo phép đếm request.

PHẢN TƯ

Độ tin cậy tăng lên nhờ ba loại bằng chứng bổ sung nhau: test-first làm lộ lỗi trước khi vá, kiểm tra bounded retry xác nhận giới hạn lời gọi, còn mutation thử xem test có thật sự phát hiện false success khi commit thất bại hay không. Mutant bị test persistence giết bằng lỗi DID NOT RAISE SQLAlchemyError; đây là bằng chứng mạnh hơn việc chỉ thấy test pass trên implementation đúng. Chạy lại toàn backend được 49 pass và Ruff pass, đồng thời đổi thứ tự các file test hai lần vẫn 15/15, giúp giảm lo ngại về state dùng chung.

Một số điều vẫn chưa được chứng minh: nhánh 404/generic 4xx và vài nhánh pagination chưa có test trực tiếp; raw event và audit append có thể tăng khi chạy lại workflow; Hypothesis ghi nhận invalid examples nhưng không ghi rõ dữ liệu gây ra; coverage giới hạn ở hai module; benchmark dùng mock và bỏ thời gian backoff thật. Vì vậy p95 0.701 ms không đại diện cho production.

Mutation testing đánh giá sức mạnh assertion bằng cách đưa một lỗi có chủ đích vào đường commit rồi xác nhận test bắt được kết quả thành công giả. Bước kế tiếp có giá trị nhất là kiểm thử integration với database cho raw-event/audit semantics khi re-run, rồi bổ sung test cho 404 và POST transport failure. Nếu có môi trường staging, đo network/API latency riêng sẽ trả lời câu hỏi hiệu năng thực tế tốt hơn harness hiện tại.

AI giúp tạo khung test và tổng hợp dữ liệu nhanh, nhưng các giả định phải được đối chiếu với code và output thật. Tôi chịu trách nhiệm chọn phạm vi, sửa oracle, giữ nguyên RED evidence, kiểm tra mutation đã gỡ, và công bố những điều chưa biết. AI hỗ trợ thao tác; kết luận kỹ thuật và nội dung nộp bài vẫn cần con người xác nhận.

Số từ phản tư (tách theo khoảng trắng): 322.

PHỤ LỤC · HỒ SƠ BẰNG CHỨNG

Phần này nằm sau nội dung chính; log gốc vẫn được lưu nguyên trong repository.

A. AI Usage Log

Nhật ký đầy đủ: kế hoạch ban đầu, quyết định review, RED/GREEN, oracle, mutation, coverage, performance và rủi ro thiếu ngữ cảnh.

Nguồn: backend/tests/evidence/vt024-ai-usage-log.md

```
# VT-024 AI Usage Log — Phase 3B patch preparation

- AI tool: OpenAI Codex (GPT-6 family; exact serving snapshot is not exposed in this task).
- Human direction: proceed with a minimal production patch after Checkpoint 3A; modify only
  `backend/app/github/client.py` unless evidence requires otherwise.
- Test-oracle correction: the retry property originally required `GitHubAPIError` after transport retry exhaustion.
  Review of the existing client showed that `httpx` transport exceptions currently propagate unchanged, and no public
  contract requires conversion to `GitHubAPIError`. The oracle now expects `httpx.TransportError`, preserving the
  error category without weakening the requirements that attempts remain bounded and exhaustion cannot return success.
- Evidence preservation: the original RED output and context in `vt024-red.txt` and `vt024-red-context.txt` remain
  unchanged. This correction is a follow-up to that evidence, not a rewrite of it.
- Hypothesis invalid examples: the property has no `assume()` or `.filter()` call. A temporary no-op probe using the
  same bounded integer/list strategy also reported invalid generated examples (3 in the probe versus 4 in the
  integration property), isolating these to Hypothesis strategy generation rather than rejected application inputs.
  The exact low-level generated byte patterns are not printed by Hypothesis statistics. The temporary probe was
  removed.
- Production decision: retry only `httpx.ConnectError` and `httpx.TimeoutException` on GET, using the existing
  exponential backoff expression. Keep the original transport exception on exhaustion. For non-GET methods, preserve
  the existing `GitHubAPIError` response behavior but do not retry transient statuses or selected transport failures.
- Not used: no AI-generated behavior was accepted without comparison to repository code and the observed RED results.
  No production changes to `BackfillService` or dependencies were proposed.
- GREEN verification: the targeted retry/backfill suite passed (14 tests), the full backend suite passed (48 tests),
  and Ruff passed after import sorting and aliasing the SQLAlchemy `event` import to avoid shadowing an existing loop
  variable. The initial Ruff run exposed those test-file issues; no unrelated production changes were made.
- Mypy: unavailable in the project `.venv` (`No module named mypy`); it was not installed during this checkpoint.
- Hypothesis GREEN statistics: bounded retry property reported 40 passing generated examples and 2 invalid strategy-
  generated examples; database idempotency property reported 30 passing examples and 4 invalid strategy-generated
  examples. Neither property reported failures or minimized counterexamples.
- Preserved limitation: the invalid-example counts are not caused by test-level `assume()` or `.filter()` calls; a
  temporary no-op probe using the same bounded/list strategy reproduced invalid generated examples. Hypothesis did not
  expose the underlying byte patterns.

## Phase 4 — Test quality and performance evaluation

- Human approval: the user approved adding `coverage` as a development dependency. Installed `coverage 7.17.3` in the
  project virtual environment and added `coverage>=7.6,<8.0` to the optional `dev` dependencies in
  `backend/pyproject.toml`. The initial sandboxed pip attempt could not access the package index; the approved
  elevated install succeeded.
- Mutation test: on a temporary local branch from GREEN commit `ecd22676f6a1ce43bcfa0d21852f717d475c130d`,
  deliberately caught `session.commit()` failure in `BackfillService.run()` and returned a fabricated `ready` result.
  The persistence regression failed with `DID NOT RAISE SQLAlchemyError`, killing the mutant as intended. Exact test
  output and unified diff are in `vt024-mutation.txt` and `vt024-mutation-diff.patch`. The mutated production source
  was restored; the assignment branch has no such mutation.
- API regression: added `test_sync_api_does_not_return_success_when_persistence_fails`. It uses a valid owner session
  cookie, an offline fake GitHub client, and an injected commit failure; it expects HTTP 500 and verifies repository
  state remains pending. The first test attempt revealed that Starlette returned a plain-text 500 body rather than
  JSON; the assertion was corrected to check the actual body, without changing production behavior.
- Performance: reviewed the task calculation and corrected an earlier erroneous scenario description. There are 90
  immediate-success tasks, 5 transient-then-success tasks, and 5 intentionally exhausted tasks. The 115 requests equal
  90×1 + 5×2 + 5×3. The refined harness uses a 100-party barrier before timing so all 100 workers are ready at the
  start; the barrier is outside the timed interval. The 100 task latencies include both success and exhaustion. The
  sleep callback is mocked to no-op, so measured time excludes actual backoff waiting. This remains an in-process mock
  experiment, not production/API/database capacity evidence.
- Coverage: the earlier text output's `Cover` column (74%, 92%, 83%) is coverage.py's combined statement-and-branch
  percentage, not line-only coverage. The machine-readable coverage JSON provides the actual statement and branch
  rates; a separate review now records those values, uncovered branches, and AC-01…AC-05 risk mapping.
- Flakiness check: reran the retry, backfill, and API tests in two different file orders; each run passed 15 tests.
  Property tests are deterministic (`derandomize=True`, no Hypothesis example database) and reported the same totals.
  No timing-based assertions or cross-task mutable state are used in the new harness/test.
- Full backend suite after Phase 4 edits: 49 passed. Ruff passed for the retry client, relevant tests, and harness.
  Mypy remains unavailable (`No module named mypy`) and was not installed. The existing Starlette/httpx TestClient
  deprecation warning remains.
- The Phase 4 context and outputs are recorded in `vt024-phase4-context.txt`, `vt024-flaky-orders.txt`, `vt024-flaky-
  review.md`, `vt024-full-suite.txt`, `vt024-ruff.txt`, `vt024-performance.txt`, `vt024-coverage-run.txt`,
  `vt024-coverage-report.txt`, `vt024-coverage.json`, and `vt024-coverage-review.md`. Original RED evidence remains
  unchanged.

## Checkpoint 6 review follow-up

- Initial implementation plan: add bounded retry for `ConnectError` and timeout failures only on GET in
  `GitHubClient`; retain the existing 403/429/5xx behavior; keep workflow-level retry out of scope; write timeout,
  connection, bounded-retry, backfill-idempotency and persistence-failure tests first. The human approved the minimal
  retry slice at Checkpoint 3A/3B and GREEN verification at Checkpoint 4.
- Human review decisions recorded in this thread: preserve the original RED logs and test-only commit; correct the
  retry oracle to allow the existing `httpx.TransportError` propagation contract; approve adding coverage.py as a dev
  dependency; require mutation evidence and a stop before report/PDF or GitHub submission.
- RED evidence: commit `11055810f10729963a8706b22fec2fcf5d9b0647` added the targeted tests first. The original run had
```

```
  four failed tests: `test_read_timeout_is_retried_then_succeeds`, `test_connection_error_is_retried_then_succeeds`,
  `test_transient_http_error_on_post_is_not_retried`, and
  `test_timeout_retry_is_bounded_and_returns_only_after_success`. The persistence characterization
  `test_commit_failure_never_returns_success_and_recovery_is_consistent` passed on the baseline. The original output
  remains unchanged in `vt024-red.txt`.
- Production patch review: commit `ecd22676f6a1ce43bcfa0d21852f717d475c130d` modifies only
  `backend/app/github/client.py` for the production retry implementation. The oracle correction is in the same commit.
  The final branch's diff of `backend/app/ingestion/backfill.py` from baseline is empty; no production change there is
  required.
- Two plausible but unreliable AI-generated test assumptions were challenged: (1) requiring `GitHubAPIError` for
  exhausted transport retries would alter the existing public error behavior, so it was changed to assert the
  propagated `httpx.TransportError`; (2) expecting a JSON body for the API's default 500 response was incorrect
  because the observed Starlette response is plain text, so the API regression asserts the actual 500 text/body. A
  third mistaken AI summary classified performance task counts as 85/10/5; source arithmetic showed 90/5/5, corrected
  here and in the harness evidence.
- Missing-context investigation: authorized inspection of `models.py` confirmed uniqueness constraints for business
  rows and sync cursors. Inspection of `BackfillService` and `AuditService` showed that raw events and audit entries
  are appended in the same unit of work and do not share the business-row upsert keys. A whole-workflow retry/re-run
  could therefore duplicate raw events and append another audit entry even though business rows remain unique. This
  supports limiting automatic retries to safe GitHub GET requests; it does not prove exact-once raw-event/audit
  semantics for manual re-runs.
- No PDF, push, PR, or merge was performed. Mypy remains unavailable and is not claimed as passing.
- Checkpoint 6 source-integrity commands and results are preserved in `vt024-mutation-removal.txt`; the acceptance-
  criterion mapping with actual function names and commit SHAs is in `vt024-traceability.md`. The separate flakiness
  analysis is in `vt024-flaky-review.md`.
- The 100-worker barrier refinement and corrected measured output are committed as
  `5f1fcbb719a8efad847e58bca03dce406566bfae`. Final Checkpoint 6 full-suite and Ruff outputs are in `vt024-final-
  tests.txt` and `vt024-final-ruff.txt`.

## Checkpoint 7B report correction

- Human review identified that the PDF had incorrectly included the passing persistence characterization among the
  four RED failures. The report now names the four actual failed tests and states separately that persistence
  characterization passed on baseline. No original RED/GREEN/mutation execution log was edited.
- Added report-only documentation IDs T01–T14 and mapped each ID to its unchanged Python test function in the risk
  table and traceability matrix.
- Added the explicit non-functional target (100 concurrent users/tasks, p95 ≤ 3.0 seconds), offline/deterministic
  testing, metadata-only privacy, bounded retries, and no unintended duplicate side effects. The mock p95 is
  explicitly not production compliance evidence.
- Only report/evidence documentation and the PDF generator are changed for this correction; production code and Python
  tests are untouched.
```

B. RED execution evidence

Log RED gốc được giữ nguyên; thể hiện test trước bản vá và các failure mong đợi.

Nguồn: backend/tests/evidence/vt024-red-context.txt

```
VT-024 RED phase context

Branch: bt3-23001819-vt024-retry
Baseline SHA: 90f831c8c8afc84dd4f0268b7ed5704e41f32cbf
Test command: .\.venv\Scripts\python.exe -m pytest -q --hypothesis-show-statistics tests/test_rate_limit.py
  tests/test_backfill.py
GitHub API access: none; HTTP interactions used httpx.MockTransport.

Environment versions:
Python 3.13.5
pytest 8.4.2
hypothesis 6.170.15
httpx 0.28.1
sqlmodel 0.0.48
fastapi 0.162.0

Actual pytest output is recorded in vt024-red.txt.
```

Nguồn: backend/tests/evidence/vt024-red.txt

```
......FFFF....                                                          [100%]
================================== FAILURES ===================================
_____________________ test_read_timeout_is_retried_then_succeeds _____________________

    def test_read_timeout_is_retried_then_succeeds() -> None:
        calls = 0

        def handler(request: httpx.Request) -> httpx.Response:
            nonlocal calls
            calls += 1
            if calls == 1:
                raise httpx.ReadTimeout("temporary timeout", request=request)
            return httpx.Response(200, json={"full_name": "duyendojhs/SE2026-T07"})

        client = GitHubClient(
            token_provider=lambda: "token",
            base_url="https://api.github.com",
            transport=httpx.MockTransport(handler),
            sleep=lambda _: None,
        )
        try:
>           payload, _ = client.get_json("/repos/duyendojhs/SE2026-T07")
                         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

tests\test_rate_limit.py:157:
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
app\github\client.py:150: in get_json
    response = self.request("GET", path, params=params, etag=etag)
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
app\github\client.py:94: in request
    response = self.client.request(
.venv\Lib\site-packages\httpx\_client.py:825: in request
    return self.send(request, auth=auth, follow_redirects=follow_redirects)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
.venv\Lib\site-packages\httpx\_client.py:914: in send
    response = self._send_handling_auth(
.venv\Lib\site-packages\httpx\_client.py:942: in _send_handling_auth
    response = self._send_handling_redirects(
.venv\Lib\site-packages\httpx\_client.py:979: in _send_handling_redirects
    response = self._send_single_request(request)
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
.venv\Lib\site-packages\httpx\_client.py:1014: in _send_single_request
    response = transport.handle_request(request)
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
.venv\Lib\site-packages\httpx\_transports\mock.py:24: in handle_request
    response = self.handler(request)
               ^^^^^^^^^^^^^^^^^^^^^
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _

request = <Request('GET', 'https://api.github.com/repos/duyendojhs/SE2026-T07')>

    def handler(request: httpx.Request) -> httpx.Response:
        nonlocal calls
        calls += 1
        if calls == 1:
>           raise httpx.ReadTimeout("temporary timeout", request=request)
E           httpx.ReadTimeout: temporary timeout

tests\test_rate_limit.py:147: ReadTimeout
_____________________ test_connection_error_is_retried_then_succeeds _____________________

    def test_connection_error_is_retried_then_succeeds() -> None:
        calls = 0

        def handler(request: httpx.Request) -> httpx.Response:
            nonlocal calls
            calls += 1
            if calls == 1:
                raise httpx.ConnectError("temporary connection error", request=request)
            return httpx.Response(200, json={"full_name": "duyendojhs/SE2026-T07"})

        client = GitHubClient(
            token_provider=lambda: "token",
            base_url="https://api.github.com",
            transport=httpx.MockTransport(handler),
            sleep=lambda _: None,
        )
        try:
>           payload, _ = client.get_json("/repos/duyendojhs/SE2026-T07")
                         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

tests\test_rate_limit.py:181:
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
app\github\client.py:150: in get_json
    response = self.request("GET", path, params=params, etag=etag)
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
app\github\client.py:94: in request
```

```
    response = self.client.request(
.venv\Lib\site-packages\httpx\_client.py:825: in request
    return self.send(request, auth=auth, follow_redirects=follow_redirects)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
.venv\Lib\site-packages\httpx\_client.py:914: in send
    response = self._send_handling_auth(
.venv\Lib\site-packages\httpx\_client.py:942: in _send_handling_auth
    response = self._send_handling_redirects(
.venv\Lib\site-packages\httpx\_client.py:979: in _send_handling_redirects
    response = self._send_single_request(request)
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
.venv\Lib\site-packages\httpx\_client.py:1014: in _send_single_request
    response = transport.handle_request(request)
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
.venv\Lib\site-packages\httpx\_transports\mock.py:24: in handle_request
    response = self.handler(request)
               ^^^^^^^^^^^^^^^^^^^^
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _

request = <Request('GET', 'https://api.github.com/repos/duyendojhs/SE2026-T07')>

    def handler(request: httpx.Request) -> httpx.Response:
        nonlocal calls
        calls += 1
        if calls == 1:
>           raise httpx.ConnectError("temporary connection error", request=request)
E           httpx.ConnectError: temporary connection error

tests\test_rate_limit.py:171: ConnectError
_____________ test_transient_http_error_on_post_is_not_retried _______________

    def test_transient_http_error_on_post_is_not_retried() -> None:
        calls = 0

        def handler(_: httpx.Request) -> httpx.Response:
            nonlocal calls
            calls += 1
            return httpx.Response(500, json={"message": "temporary server failure"})

        client = GitHubClient(
            token_provider=lambda: "token",
            base_url="https://api.github.com",
            transport=httpx.MockTransport(handler),
            sleep=lambda _: None,
        )
        try:
            with pytest.raises(GitHubAPIError):
                client.request("POST", "/repos/duyendojhs/SE2026-T07")
>           assert calls == 1
E           assert 4 == 1

tests\test_rate_limit.py:205: AssertionError
________ test_timeout_retry_is_bounded_and_returns_only_after_success _________
  + Exception Group Traceback (most recent call last):
  |   File "C:\HUS\SE\Prj_AI03\SE2026-T07\backend\tests\test_rate_limit.py", line 211, in
  test_timeout_retry_is_bounded_and_returns_only_after_success
  |     max_retries=st.integers(min_value=0, max_value=5),
  |                 ^^^
  |   File "C:\HUS\SE\Prj_AI03\SE2026-T07\backend\.venv\Lib\site-packages\hypothesis\core.py", line 2334, in
  wrapped_test
  |     raise the_error_hypothesis_found
  | ExceptionGroup: Hypothesis found 2 distinct failures. (2 sub-exceptions)
  +-+---------------- 1 ----------------
    | Traceback (most recent call last):
    |   File "C:\HUS\SE\Prj_AI03\SE2026-T07\backend\tests\test_rate_limit.py", line 246, in
  test_timeout_retry_is_bounded_and_returns_only_after_success
    |     client.get_json("/repos/duyendojhs/SE2026-T07")
    |     ~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
    |   File "C:\HUS\SE\Prj_AI03\SE2026-T07\backend\app\github\client.py", line 150, in get_json
    |     response = self.request("GET", path, params=params, etag=etag)
    |   File "C:\HUS\SE\Prj_AI03\SE2026-T07\backend\app\github\client.py", line 94, in request
    |     response = self.client.request(
    |         method,
    |     ...<2 lines>...
    |         headers=headers,
    |     )
    |   File "C:\HUS\SE\Prj_AI03\SE2026-T07\backend\.venv\Lib\site-packages\httpx\_client.py", line 825, in request
    |     return self.send(request, auth=auth, follow_redirects=follow_redirects)
    |            ~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
    |   File "C:\HUS\SE\Prj_AI03\SE2026-T07\backend\.venv\Lib\site-packages\httpx\_client.py", line 914, in send
    |     response = self._send_handling_auth(
    |         request,
    |     ...<2 lines>...
    |         history=[],
    |     )
    |   File "C:\HUS\SE\Prj_AI03\SE2026-T07\backend\.venv\Lib\site-packages\httpx\_client.py", line 942, in
  _send_handling_auth
    |     response = self._send_handling_redirects(
    |         request,
```

```
    |         follow_redirects=follow_redirects,
    |         history=history,
    |     )
    |   File "C:\HUS\SE\Prj_AI03\SE2026-T07\backend\.venv\Lib\site-packages\httpx\_client.py", line 979, in
  _send_handling_redirects
    |     response = self._send_single_request(request)
    |   File "C:\HUS\SE\Prj_AI03\SE2026-T07\backend\.venv\Lib\site-packages\httpx\_client.py", line 1014, in
  _send_single_request
    |     response = transport.handle_request(request)
    |   File "C:\HUS\SE\Prj_AI03\SE2026-T07\backend\.venv\Lib\site-packages\httpx\_transports\mock.py", line 24, in
  handle_request
    |     response = self.handler(request)
    |   File "C:\HUS\SE\Prj_AI03\SE2026-T07\backend\tests\test_rate_limit.py", line 229, in handler
    |     raise httpx.ConnectError("temporary connection error", request=request)
    | httpx.ConnectError: temporary connection error
    | Failing test case: test_timeout_retry_is_bounded_and_returns_only_after_success(
    |     max_retries=0,  # or any other generated value
    |     transient_failures=['connect'],
    | )
    | Explanation:
    |     These lines were always and only run by failing test cases:
    |         C:\HUS\SE\Prj_AI03\SE2026-T07\backend\tests\test_rate_limit.py:226
    +---------------- 2 ----------------
    | Traceback (most recent call last):
    |   File "C:\HUS\SE\Prj_AI03\SE2026-T07\backend\tests\test_rate_limit.py", line 246, in
  test_timeout_retry_is_bounded_and_returns_only_after_success
    |     client.get_json("/repos/duyendojhs/SE2026-T07")
    |     ~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
    |   File "C:\HUS\SE\Prj_AI03\SE2026-T07\backend\app\github\client.py", line 150, in get_json
    |     response = self.request("GET", path, params=params, etag=etag)
    |   File "C:\HUS\SE\Prj_AI03\SE2026-T07\backend\app\github\client.py", line 94, in request
    |     response = self.client.request(
    |         method,
    |     ...<2 lines>...
    |         headers=headers,
    |     )
    |   File "C:\HUS\SE\Prj_AI03\SE2026-T07\backend\.venv\Lib\site-packages\httpx\_client.py", line 825, in request
    |     return self.send(request, auth=auth, follow_redirects=follow_redirects)
    |            ~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
    |   File "C:\HUS\SE\Prj_AI03\SE2026-T07\backend\.venv\Lib\site-packages\httpx\_client.py", line 914, in send
    |     response = self._send_handling_auth(
    |         request,
    |     ...<2 lines>...
    |         history=[],
    |     )
    |   File "C:\HUS\SE\Prj_AI03\SE2026-T07\backend\.venv\Lib\site-packages\httpx\_client.py", line 942, in
  _send_handling_auth
    |     response = self._send_handling_redirects(
    |         request,
    |         follow_redirects=follow_redirects,
    |         history=history,
    |     )
    |   File "C:\HUS\SE\Prj_AI03\SE2026-T07\backend\.venv\Lib\site-packages\httpx\_client.py", line 979, in
  _send_handling_redirects
    |     response = self._send_single_request(request)
    |   File "C:\HUS\SE\Prj_AI03\SE2026-T07\backend\.venv\Lib\site-packages\httpx\_client.py", line 1014, in
  _send_single_request
    |     response = transport.handle_request(request)
    |   File "C:\HUS\SE\Prj_AI03\SE2026-T07\backend\.venv\Lib\site-packages\httpx\_transports\mock.py", line 24, in
  handle_request
    |     response = self.handler(request)
    |   File "C:\HUS\SE\Prj_AI03\SE2026-T07\backend\tests\test_rate_limit.py", line 228, in handler
    |     raise httpx.ReadTimeout("temporary timeout", request=request)
    | httpx.ReadTimeout: temporary timeout
    | Failing test case: test_timeout_retry_is_bounded_and_returns_only_after_success(
    |     max_retries=0,  # or any other generated value
    |     transient_failures=['timeout'],
    | )
    | Explanation:
    |     These lines were always and only run by failing test cases:
    |         C:\HUS\SE\Prj_AI03\SE2026-T07\backend\tests\test_rate_limit.py:226
    +-----------------------------------
============================== warnings summary ===============================
.venv\Lib\site-packages\fastapi\testclient.py:1
  C:\HUS\SE\Prj_AI03\SE2026-T07\backend\.venv\Lib\site-packages\fastapi\testclient.py:1: StarletteDeprecationWarning:
  Using `httpx` with `starlette.testclient` is deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
=========================== Hypothesis Statistics ============================
tests/test_rate_limit.py::test_timeout_retry_is_bounded_and_returns_only_after_success:

  - during generate phase (0.15 seconds):
    - Typical runtimes: ~ 0-81 ms, of which ~ 0-5 ms in data generation
    - 3 passing, 8 failing, and 3 invalid test cases
    - Found 2 distinct errors in this phase

  - during shrink phase (0.03 seconds):
    - Typical runtimes: ~ 2-5 ms, of which < 1ms in data generation
```

```
    - 1 passing, 4 failing, and 0 invalid test cases
    - Tried 5 shrinks of which 4 were successful

  - during explain phase (0.05 seconds):
    - Typical runtimes: ~ 2ms, of which < 1ms in data generation
    - 0 passing, 8 failing, and 0 invalid test cases

  - Stopped because nothing left to do


tests/test_backfill.py::test_generated_duplicate_issue_ids_remain_unique_after_reordered_reingestion:

  - during generate phase (3.68 seconds):
    - Typical runtimes: ~ 0-147 ms, of which ~ 0-1 ms in data generation
    - 30 passing, 0 failing, and 4 invalid test cases

  - Stopped because settings.max_examples=30


=========================== short test summary info ============================
FAILED tests/test_rate_limit.py::test_read_timeout_is_retried_then_succeeds
FAILED tests/test_rate_limit.py::test_connection_error_is_retried_then_succeeds
FAILED tests/test_rate_limit.py::test_transient_http_error_on_post_is_not_retried
FAILED tests/test_rate_limit.py::test_timeout_retry_is_bounded_and_returns_only_after_success
```

C. GREEN execution evidence

Bao gồm targeted GREEN, Hypothesis statistics, full suite cuối, Ruff và limitation mypy.

Nguồn: backend/tests/evidence/vt024-green-context.txt

```
VT-024 GREEN verification context

Branch: bt3-23001819-vt024-retry
Baseline SHA: 90f831c8c8afc84dd4f0268b7ed5704e41f32cbf
HEAD before the production commit: 11055810f10729963a8706b22fec2fcf5d9b0647 (test-only RED commit)
Original test-only commit: 11055810f10729963a8706b22fec2fcf5d9b0647
The production patch, oracle correction, and evidence were uncommitted while GREEN commands ran.
Temporary trace_invalid_tmp.py: absent and not tracked.

Environment versions:
Python 3.13.5
pytest 8.4.2
Hypothesis 6.170.15
httpx 0.28.1
SQLModel 0.0.48
FastAPI 0.162.0
Ruff 0.17.3
mypy: unavailable in .venv (`python -m mypy app` returned `No module named mypy`)

Observed results:
- Targeted retry/backfill suite: 14 passed.
- Full backend suite: 48 passed.
- Ruff: passed on final modified source/test files. An earlier Ruff invocation found import sorting and a test import-
  name collision; those test-file issues were fixed, then Ruff passed.
- Hypothesis retry property: 40 passing generated examples, 2 invalid, 0 failing/minimized counterexamples.
- Hypothesis backfill idempotency property: 30 passing generated examples, 4 invalid, 0 failing/minimized
  counterexamples.
- Pytest emitted a Starlette deprecation warning about using httpx with TestClient; tests passed.
- No GitHub API calls were made.

Residual risks reviewed:
- `httpx.PoolTimeout` is covered by `httpx.TimeoutException`; retrying it can add bounded delay when local connection-
  pool capacity, rather than GitHub, is the bottleneck.
- `httpx.WriteTimeout` is also covered. The retry is restricted to GET, so a partially transmitted request cannot
  repeat a business write; GET may still consume extra read capacity.
- HTTP 403 handling for GET is intentionally preserved, including `Retry-After` and bounded fallback backoff. A
  permission-related 403 may be permanent, so retries can add bounded delay without resolving it. Non-GET 403
  responses fail immediately with `GitHubAPIError`.

Exact commands and terminal outputs are in vt024-green.txt.
```

Nguồn: backend/tests/evidence/vt024-green.txt

```
COMMAND: .\.venv\Scripts\python.exe -m pytest -q --hypothesis-show-statistics tests/test_rate_limit.py
  tests/test_backfill.py
.............                                                            [100%]
============================== warnings summary ===============================
.venv\Lib\site-packages\fastapi\testclient.py:1
  C:\HUS\SE\Prj_AI03\SE2026-T07\backend\.venv\Lib\site-packages\fastapi\testclient.py:1: StarletteDeprecationWarning:
  Using `httpx` with `starlette.testclient` is deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
============================ Hypothesis Statistics ============================
tests/test_rate_limit.py::test_timeout_retry_is_bounded_and_returns_only_after_success:

  - during generate phase (0.16 seconds):
    - Typical runtimes: ~ 0-2 ms, of which < 1ms in data generation
    - 40 passing, 0 failing, and 2 invalid test cases

  - Stopped because settings.max_examples=40


tests/test_backfill.py::test_generated_duplicate_issue_ids_remain_unique_after_reordered_reingestion:

  - during generate phase (2.41 seconds):
    - Typical runtimes: ~ 0-96 ms, of which < 1ms in data generation
    - 30 passing, 0 failing, and 4 invalid test cases

  - Stopped because settings.max_examples=30


EXIT CODE: 0

COMMAND: .\.venv\Scripts\python.exe -m pytest -q
.............................................                            [100%]
============================== warnings summary ===============================
.venv\Lib\site-packages\fastapi\testclient.py:1
  C:\HUS\SE\Prj_AI03\SE2026-T07\backend\.venv\Lib\site-packages\fastapi\testclient.py:1: StarletteDeprecationWarning:
  Using `httpx` with `starlette.testclient` is deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
EXIT CODE: 0

COMMAND: .\.venv\Scripts\ruff.exe check app/github/client.py tests/test_rate_limit.py tests/test_backfill.py
I001 [*] Import block is un-sorted or un-formatted
  --> tests\test_backfill.py:1:1
   |
 1 | / from collections.abc import Iterator
 2 | | from typing import Any
 3 | |
 4 | | import pytest
 5 | | from hypothesis import given, settings, strategies as st
 6 | | from sqlalchemy import event
 7 | | from sqlalchemy.exc import SQLAlchemyError
 8 | | from sqlalchemy.pool import StaticPool
 9 | | from sqlmodel import Session, SQLModel, create_engine, select
10 | |
11 | | from app.ingestion.backfill import BackfillService
12 | | from app.models import (
13 | |     Comment,
14 | |     Commit,
15 | |     Installation,
16 | |     Issue,
17 | |     PullRequest,
18 | |     RawEvent,
19 | |     Repository,
20 | |     Review,
21 | |     Tenant,
22 | | )
   | |_^
help: Organize imports
  |
4 | import pytest
  - from hypothesis import given, settings, strategies as st
5 + from hypothesis import given, settings
6 + from hypothesis import strategies as st
7 | from sqlalchemy import event
  |

F402 Import `event` from line 6 shadowed by loop variable
   --> tests\test_backfill.py:158:9
    |
156 |     raw_events = session.exec(select(RawEvent)).all()
157 |     assert raw_events
158 |     for event in raw_events:
    |         ^^^^^
159 |         assert "body" not in event.payload_json
160 |         assert "patch" not in event.payload_json
```

```
    |

I001 [*] Import block is un-sorted or un-formatted
 --> tests\test_rate_limit.py:1:1
  |
1 | / import httpx
2 | | import pytest
3 | | from hypothesis import given, settings, strategies as st
4 | |
5 | | from app.github.allowed_endpoints import EndpointNotAllowed
6 | | from app.github.app_auth import GitHubAPIError
7 | | from app.github.client import GitHubClient
  | |_____________________________________________^
help: Organize imports
  |
2 | import pytest
  - from hypothesis import given, settings, strategies as st
3 + from hypothesis import given, settings
4 + from hypothesis import strategies as st
5 |
  |

Found 3 errors.
[*] 2 fixable with `--fix`.
EXIT CODE: 1

COMMAND (after Ruff import fix): .\.venv\Scripts\python.exe -m pytest -q --hypothesis-show-statistics
  tests/test_rate_limit.py tests/test_backfill.py
.............                                                            [100%]
============================== warnings summary ===============================
.venv\Lib\site-packages\fastapi\testclient.py:1
  C:\HUS\SE\Prj_AI03\SE2026-T07\backend\.venv\Lib\site-packages\fastapi\testclient.py:1: StarletteDeprecationWarning:
  Using `httpx` with `starlette.testclient` is deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
=========================== Hypothesis Statistics ============================
tests/test_rate_limit.py::test_timeout_retry_is_bounded_and_returns_only_after_success:

  - during generate phase (0.18 seconds):
    - Typical runtimes: ~ 0-3 ms, of which < 1ms in data generation
    - 40 passing, 0 failing, and 2 invalid test cases

  - Stopped because settings.max_examples=40


tests/test_backfill.py::test_generated_duplicate_issue_ids_remain_unique_after_reordered_reingestion:

  - during generate phase (2.51 seconds):
    - Typical runtimes: ~ 0-100 ms, of which ~ 0-1 ms in data generation
    - 30 passing, 0 failing, and 4 invalid test cases

  - Stopped because settings.max_examples=30


EXIT CODE: 0

COMMAND (after Ruff import fix): .\.venv\Scripts\python.exe -m pytest -q
.............................................                            [100%]
============================== warnings summary ===============================
.venv\Lib\site-packages\fastapi\testclient.py:1
  C:\HUS\SE\Prj_AI03\SE2026-T07\backend\.venv\Lib\site-packages\fastapi\testclient.py:1: StarletteDeprecationWarning:
  Using `httpx` with `starlette.testclient` is deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
EXIT CODE: 0

COMMAND (after import fix): .\.venv\Scripts\ruff.exe check app/github/client.py tests/test_rate_limit.py
  tests/test_backfill.py
All checks passed!
EXIT CODE: 0

COMMAND: .\.venv\Scripts\python.exe -m mypy app
C:\HUS\SE\Prj_AI03\SE2026-T07\backend\.venv\Scripts\python.exe: No module named mypy
EXIT CODE: 1
```

Nguồn: backend/tests/evidence/vt024-final-tests.txt

```
.............................................                            [100%]
============================== warnings summary ===============================
.venv\Lib\site-packages\fastapi\testclient.py:1
  C:\HUS\SE\Prj_AI03\SE2026-T07\backend\.venv\Lib\site-packages\fastapi\testclient.py:1: StarletteDeprecationWarning:
  Using `httpx` with `starlette.testclient` is deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
49 passed, 1 warning in 3.15s

Exit code: 0
```

Nguồn: backend/tests/evidence/vt024-final-ruff.txt

```
All checks passed!

Exit code: 0
```

Nguồn: backend/tests/evidence/vt024-mypy.txt

```
C:\HUS\SE\Prj_AI03\SE2026-T07\backend\.venv\Scripts\python.exe: No module named mypy

Exit code: 1
```

D. Mutation diff and kill output

Mutant chỉ được dùng trong thí nghiệm; bản vá và output thất bại của regression được lưu.

Nguồn: backend/tests/evidence/vt024-mutation.txt

```
F                                                                        [100%]
================================== FAILURES ===================================
____ test_commit_failure_never_returns_success_and_recovery_is_consistent _____

session = <sqlmodel.orm.session.Session object at 0x000001E43AACD550>

    def test_commit_failure_never_returns_success_and_recovery_is_consistent(
        session: Session,
    ) -> None:
        tenant, repo = _seed(session)

        def fail_commit(_: Session) -> None:
            raise SQLAlchemyError("injected commit failure")

        sqlalchemy_event.listen(session, "before_commit", fail_commit)
        try:
>           with pytest.raises(SQLAlchemyError, match="injected commit failure"):
                 ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
E           Failed: DID NOT RAISE <class 'sqlalchemy.exc.SQLAlchemyError'>

tests\test_backfill.py:285: Failed
============================== warnings summary ===============================
.venv\Lib\site-packages\fastapi\testclient.py:1
  C:\HUS\SE\Prj_AI03\SE2026-T07\backend\.venv\Lib\site-packages\fastapi\testclient.py:1: StarletteDeprecationWarning:
  Using `httpx` with `starlette.testclient` is deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
=========================== short test summary info ===========================
FAILED tests/test_backfill.py::test_commit_failure_never_returns_success_and_recovery_is_consistent

Exit code: 1
```

Nguồn: backend/tests/evidence/vt024-mutation-diff.patch

```
diff --git a/backend/app/ingestion/backfill.py b/backend/app/ingestion/backfill.py
index 877db3c..b33bf52 100644
--- a/backend/app/ingestion/backfill.py
+++ b/backend/app/ingestion/backfill.py
@@ -110,7 +110,10 @@ class BackfillService:
             self.repo.sync_status = "ready"
             self.repo.sync_error = None
         self.session.add(self.repo)
-        self.session.commit()
+        try:
+            self.session.commit()
+        except Exception:  # deliberate VT-024 mutation
+            return BackfillResult(status="ready", counts=self.counts, errors=[])
         return BackfillResult(status=self.repo.sync_status, counts=self.counts, errors=self.errors)

     def _ingest_issues(self, since: datetime) -> set[int]:
```

Nguồn: backend/tests/evidence/vt024-mutation-removal.txt

```
VT-024 mutation-removal verification

Branch: bt3-23001819-vt024-retry
Baseline: 90f831c8c8afc84dd4f0268b7ed5704e41f32cbf
HEAD at the Checkpoint 6 verification start: 5f1fcbb719a8efad847e58bca03dce406566bfae

Commands run:
1. `git status --short`
2. `git diff 90f831c8c8afc84dd4f0268b7ed5704e41f32cbf HEAD -- backend/app/ingestion/backfill.py`
3. `git show --stat e4e56c86b9dab3e3e3c2fbaf36b751f29e4b135a`

Observed output:
- `git status --short`: empty at the start of the review.
- Baseline-to-HEAD diff for `backend/app/ingestion/backfill.py`: empty.
- `git show --stat e4e56c86b9dab3e3e3c2fbaf36b751f29e4b135a`: 15 files changed, 393 insertions; the file list contains
  `backend/pyproject.toml`, tests, harness, and evidence files; it does not contain
  `backend/app/ingestion/backfill.py`.
- The production `backfill.py` is unchanged on the assignment branch. Its only mutation text is preserved in the
  evidence patch `vt024-mutation-diff.patch`; the deliberate mutation is not in application source.
- The mutation-killing test output remains preserved in `vt024-mutation.txt`.

Note: the command SHA in item 3 is `e4e56c86b9dab3e3e3c2fbaf36b751f29e4b135a` (the quality-evidence commit).
```

E. Coverage report and risk mapping

Coverage giới hạn ở client và backfill; review ghi rõ statement, branch, combined và nhánh thiếu.

Nguồn: backend/tests/evidence/vt024-coverage-run.txt

```
..............                                                           [100%]
============================== warnings summary ===============================
.venv\Lib\site-packages\fastapi\testclient.py:1
  C:\HUS\SE\Prj_AI03\SE2026-T07\backend\.venv\Lib\site-packages\fastapi\testclient.py:1: StarletteDeprecationWarning:
  Using `httpx` with `starlette.testclient` is deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html

Exit code: 0
```

Nguồn: backend/tests/evidence/vt024-coverage-report.txt

```
Name                        Stmts   Miss Branch BrPart  Cover   Missing
-----------------------------------------------------------------------
app\github\client.py          121     26     50     15    74%   58-61, 63-66, 71-75, 88, 111, 127-130, 143, 149, 160,
  182, 185, 189, 203, 205->200, 207
app\ingestion\backfill.py     163     12     40      5    92%   82-83, 87-88, 99-100, 107-108, 130, 154, 173, 302
-----------------------------------------------------------------------
TOTAL                         284     38     90     20    83%

Exit code: 0
```

Nguồn: backend/tests/evidence/vt024-coverage-review.md

````
# VT-024 Coverage and Risk Review

## Commands and scope

Coverage was collected for only the selected GitHub retry/backfill vertical slice, not the repository:

```powershell
.\.venv\Scripts\python.exe -m coverage run --branch --source=app.github.client,app.ingestion.backfill -m pytest -q
  tests/test_rate_limit.py tests/test_backfill.py tests/test_tenants_sync_api.py
.\.venv\Scripts\python.exe -m coverage report -m
.\.venv\Scripts\python.exe -m coverage json -o tests/evidence/vt024-coverage.json
```

The selected test run passed 15 tests. Raw terminal outputs are in `vt024-coverage-run.txt` and `vt024-coverage-
  report.txt`; the machine-readable branch details are in `vt024-coverage.json`.

## Correct interpretation of coverage.py metrics

The text report's `Cover` column includes statements and branch arcs; its 74%, 92%, and 83% values are **combined
  coverage**, not line-only coverage. The JSON separates statement and branch rates:

| Production module | Covered statements / statements | Statement coverage | Covered branches / branches | Branch
  coverage | Combined coverage.py `Cover` |
|---|---:|---:|---:|---:|---:|
| `app/github/client.py` | 95 / 121 | 78.51% (79%) | 31 / 50 | 62.00% | 73.68% (74%) |
| `app/ingestion/backfill.py` | 151 / 163 | 92.64% (93%) | 35 / 40 | 87.50% (88%) | 91.63% (92%) |
| Selected-module total | 246 / 284 | 86.62% (87%) | 66 / 90 | 73.33% (73%) | 83.42% (83%) |

`covered_lines` in coverage.py JSON is the count of executed statements; the displayed line/statement rate is computed
  against executable statements. Twenty branch decision points are partial; they contribute 24 missing branch arcs.
  This is not full-repository coverage.

## Uncovered paths relevant to risk

- `GitHubClient`: safe GET retries for 5xx, `ConnectError`, and timeout and the bounded exhaustion path are exercised.
  Missing branch arcs include rate-reset waiting (`client.py:69-75`), absent/malformed `Retry-After` fallback
  (`123-130`), generic 4xx propagation (`142-146`), 304 handling (`110`, `159`, `181`), and malformed/non-list
  pagination cases (`184-189`, `200-207`). There is no distinct direct test for 404 or 429, though 401 and 403 cases
  exercise their shared code branches.
- `BackfillService`: partial failure with some ingested counts is tested. The all-resources-fail path that yields
  `error` rather than `partial` (`backfill.py:106-108`) is not directly covered. Remaining missing arcs include
  skipping an issue that is also a PR (`129-130`), stale PR cutoff (`153-154`), and missing PR during review ingestion
  (`172-173`).
- These gaps are not hidden by the selected-module percentages. The safe retry and persistence invariants have direct
  targeted tests; unrelated client utility paths remain uncovered by this slice.

## Acceptance-criterion risk coverage

| Criterion | Evidence exercised | Risk coverage conclusion |
|---|---|---|
| AC-01 transient 5xx/connection/timeout retries within a configured bound | `test_http_500_is_retried_then_succeeds`,
  `test_read_timeout_is_retried_then_succeeds`, `test_connection_error_is_retried_then_succeeds`, and bounded retry
  property | Strong for selected GET transport/status cases. No live network behavior is claimed. |
| AC-02 immediate propagation for non-retryable errors and failure on exhaustion | `test_bad_token_is_not_retried`,
  `test_transient_http_error_on_post_is_not_retried`, `test_rate_limit_exhaustion_raises`, bounded retry property |
  Good for 401, POST/500, 403 exhaustion, and transport exhaustion. 404, generic 4xx, and POST transport-error cases
  lack distinct direct tests. |
| AC-03 repeated metadata does not duplicate business records/side effects |
  `test_backfill_is_idempotent_and_metadata_only`,
  `test_generated_duplicate_issue_ids_remain_unique_after_reordered_reingestion` | Business-row uniqueness is
  exercised. RawEvent and audit append counts are not asserted; repeated whole backfills can append additional
  RawEvent/audit records, so exact-once secondary side effects remain unproven and are a known risk. |
| AC-04 persistence failure never yields success |
  `test_commit_failure_never_returns_success_and_recovery_is_consistent`,
  `test_sync_api_does_not_return_success_when_persistence_fails`, deliberate false-success mutant | Strong for service
  result and API response. The mutant was killed by the commit-failure regression. |
| AC-05 partial ingestion preserves consistency/status | `test_backfill_marks_partial_when_one_resource_fails` |
  Partial status and surviving business rows are directly exercised. All-resources-failed status and cross-resource
  rollback behavior are not separately covered. |

## Mutation and source-integrity cross-check

At Checkpoint 6, the assignment branch has no working-tree changes; `git diff 90f831c8c8afc84dd4f0268b7ed5704e41f32cbf
  HEAD -- backend/app/ingestion/backfill.py` is empty. The intentional mutation is preserved only as evidence in
  `vt024-mutation-diff.patch`; it is not present in the branch source. The RED, GREEN, and quality-evidence commits
  remain in history.
````

F. Flaky-test evidence

Hai thứ tự test, chính sách time/backoff, Hypothesis invalid counts và giới hạn đã quan sát.

Nguồn: backend/tests/evidence/vt024-flaky-orders.txt

```
VT-024 order-repeat checks

COMMAND: .\.venv\Scripts\python.exe -m pytest -q tests/test_rate_limit.py tests/test_backfill.py
  tests/test_tenants_sync_api.py
..............                                                           [100%]
============================== warnings summary ===============================
.venv\Lib\site-packages\fastapi\testclient.py:1
  C:\HUS\SE\Prj_AI03\SE2026-T07\backend\.venv\Lib\site-packages\fastapi\testclient.py:1: StarletteDeprecationWarning:
  Using `httpx` with `starlette.testclient` is deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
Exit code: 0

COMMAND: .\.venv\Scripts\python.exe -m pytest -q tests/test_backfill.py tests/test_tenants_sync_api.py
  tests/test_rate_limit.py
..............                                                           [100%]
============================== warnings summary ===============================
.venv\Lib\site-packages\fastapi\testclient.py:1
  C:\HUS\SE\Prj_AI03\SE2026-T07\backend\.venv\Lib\site-packages\fastapi\testclient.py:1: StarletteDeprecationWarning:
  Using `httpx` with `starlette.testclient` is deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
Exit code: 0
```

Nguồn: backend/tests/evidence/vt024-flaky-review.md

```
# VT-024 Flaky-Test Review

## Time, sleep, and backoff

- `test_rate_limit_retry_after_is_respected` uses a fake sleep callback that records the requested delay and asserts
  the configured `Retry-After` value; it does not wait one second.
- Timeout, connection-error, HTTP 5xx, and bounded property tests use `sleep=lambda _: None`, so no real backoff delay
  is part of their pass/fail behavior.
- `GitHubClient` adds random jitter to fallback backoff, but the tests do not assert that random value and their sleep
  callback is a no-op. This avoids wall-clock flakes while leaving production timing/backoff duration unmeasured by
  those tests.
- The timing test uses monotonic `time.perf_counter` only for reporting; there is no latency threshold assertion in
  pytest.

## Hypothesis inputs and shrinking

- Retry property: `@settings(max_examples=40, derandomize=True, database=None)`; result was 40 passing, 2 invalid, 0
  failing.
- Idempotency property: `@settings(max_examples=30, derandomize=True, database=None)`; result was 30 passing, 4
  invalid, 0 failing.
- Neither test has `assume()` or `.filter()`. A temporary no-op probe using the same bounded strategy also produced
  invalid generated examples (3 in that probe). The production test bodies do not report a minimized failure.
  Hypothesis's captured statistics do not identify the serialized data for each invalid example, so the exact internal
  generation cause remains unresolved.

## Test order and shared state

- Ran both `tests/test_rate_limit.py tests/test_backfill.py tests/test_tenants_sync_api.py` and the reversed order
  `tests/test_backfill.py tests/test_tenants_sync_api.py tests/test_rate_limit.py`; each run passed 15 tests. Exact
  commands/output are in `vt024-flaky-orders.txt`.
- Pytest database fixtures create an in-memory SQLite engine per test. The property test also creates/drops its own
  isolated engine.
- The API failure test removes its `get_session` dependency override and SQLAlchemy event listener in `finally`; the
  full suite passed after the API test was included.

## Concurrency and external services

- Tests use `httpx.MockTransport` or local fake clients; no GitHub API calls are made, so network variability is not
  part of automated results.
- The performance harness runs 100 executor workers and waits for all 100 at a barrier before measured work. Each task
  owns its client, mock handler, and attempt counter. One mock run passed with 95 success outcomes and 5 intentionally
  exhausted outcomes. Mock latency is not a substitute for concurrent API/database or real network testing.
```

Nguồn: backend/tests/evidence/vt024-hypothesis-final.txt

```
..                                                                       [100%]
============================== warnings summary ===============================
.venv\Lib\site-packages\fastapi\testclient.py:1
  C:\HUS\SE\Prj_AI03\SE2026-T07\backend\.venv\Lib\site-packages\fastapi\testclient.py:1: StarletteDeprecationWarning:
  Using `httpx` with `starlette.testclient` is deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
============================ Hypothesis Statistics ============================
tests/test_rate_limit.py::test_timeout_retry_is_bounded_and_returns_only_after_success:

  - during generate phase (0.20 seconds):
    - Typical runtimes: ~ 1-3 ms, of which < 1ms in data generation
    - 40 passing, 0 failing, and 2 invalid test cases

  - Stopped because settings.max_examples=40


tests/test_backfill.py::test_generated_duplicate_issue_ids_remain_unique_after_reordered_reingestion:

  - during generate phase (2.97 seconds):
    - Typical runtimes: ~ 0-137 ms, of which ~ 0-1 ms in data generation
    - 30 passing, 0 failing, and 4 invalid test cases

  - Stopped because settings.max_examples=30


Exit code: 0
```

G. Performance harness and output

Harness có barrier 100 worker; latency nearest-rank tính trên toàn bộ outcome, gồm 5 exhausted.

Nguồn: backend/tests/perf_retry_harness.py

```
"""Mocked concurrent retry harness; not a production throughput benchmark."""

from __future__ import annotations

import json
import math
import time
from concurrent.futures import ThreadPoolExecutor
from threading import Barrier
from typing import Any

import httpx

from app.github.app_auth import GitHubAPIError
from app.github.client import GitHubClient

TASK_COUNT = 100
MAX_RETRIES = 2
REPO_PATH = "/repos/duyendojhs/SE2026-T07"


def _nearest_rank(values: list[float], percentile: float) -> float:
    ordered = sorted(values)
    return ordered[max(0, math.ceil(percentile * len(ordered)) - 1)]


def _run_task(task_id: int, start_barrier: Barrier) -> tuple[float, str, int]:
    """Run one task with entirely task-local transport and call state."""
    calls = 0
    persistent_failure = task_id % 20 == 0
    one_transient_failure = task_id % 10 == 0 and not persistent_failure

    def handler(_: httpx.Request) -> httpx.Response:
        nonlocal calls
        calls += 1
        if persistent_failure or (one_transient_failure and calls == 1):
            return httpx.Response(503, json={"message": "mock transient failure"})
        return httpx.Response(200, json={"ok": True, "task_id": task_id})

    client = GitHubClient(
        token_provider=lambda: "mock-token",
        base_url="https://api.github.com",
        transport=httpx.MockTransport(handler),
        sleep=lambda _: None,
        max_retries=MAX_RETRIES,
    )
    # Ensure all worker tasks are ready before the measured requests begin.
    start_barrier.wait()
    started = time.perf_counter()
    try:
        client.get_json(REPO_PATH)
    except GitHubAPIError:
        outcome = "exhausted"
    else:
        outcome = "success"
    finally:
        elapsed_ms = (time.perf_counter() - started) * 1000
        client.close()
    return elapsed_ms, outcome, calls


def main() -> None:
    start_barrier = Barrier(TASK_COUNT)
    with ThreadPoolExecutor(max_workers=TASK_COUNT) as executor:
        futures = [
            executor.submit(_run_task, task_id, start_barrier)
            for task_id in range(TASK_COUNT)
        ]
        results = [future.result() for future in futures]

    latencies = [latency for latency, _, _ in results]
    outcomes = [outcome for _, outcome, _ in results]
    request_classes = {
        "immediate_success_tasks": sum(
            1 for task_id in range(TASK_COUNT) if task_id % 10 != 0
        ),
        "transient_then_success_tasks": sum(
            1
            for task_id in range(TASK_COUNT)
            if task_id % 10 == 0 and task_id % 20 != 0
        ),
        "injected_exhaustion_tasks": sum(
            1 for task_id in range(TASK_COUNT) if task_id % 20 == 0
        ),
    }
    report: dict[str, Any] = {
        "harness": "mocked GitHubClient calls; no network or database",
```

```
        "tasks": TASK_COUNT,
        "workers": TASK_COUNT,
        "concurrency_gate": "all 100 workers wait at a Barrier before timing starts",
        "max_retries": MAX_RETRIES,
        "injected_scenarios": request_classes,
        "results": {
            "success": outcomes.count("success"),
            "exhausted": outcomes.count("exhausted"),
            "error_rate_percent": round(100 * outcomes.count("exhausted") / TASK_COUNT, 2),
            "attempts_total": sum(attempts for _, _, attempts in results),
        },
        "latency_ms": {
            "p50": round(_nearest_rank(latencies, 0.50), 3),
            "p95": round(_nearest_rank(latencies, 0.95), 3),
            "max": round(max(latencies), 3),
            "population": "all 100 outcomes, including exhausted tasks",
            "method": "nearest-rank percentile over per-task perf_counter durations",
        },
        "state_isolation": "each worker owns its client, MockTransport, and attempt counter",
    }
    print(json.dumps(report, indent=2, sort_keys=True))


if __name__ == "__main__":
    main()
```

Nguồn: backend/tests/evidence/vt024-performance.txt

```
{
  "concurrency_gate": "all 100 workers wait at a Barrier before timing starts",
  "harness": "mocked GitHubClient calls; no network or database",
  "injected_scenarios": {
    "immediate_success_tasks": 90,
    "injected_exhaustion_tasks": 5,
    "transient_then_success_tasks": 5
  },
  "latency_ms": {
    "max": 1.131,
    "method": "nearest-rank percentile over per-task perf_counter durations",
    "p50": 0.376,
    "p95": 0.701,
    "population": "all 100 outcomes, including exhausted tasks"
  },
  "max_retries": 2,
  "results": {
    "attempts_total": 115,
    "error_rate_percent": 5.0,
    "exhausted": 5,
    "success": 95
  },
  "state_isolation": "each worker owns its client, MockTransport, and attempt counter",
  "tasks": 100,
  "workers": 100
}

Exit code: 0
```

H. Commands, traceability and commit history

Các lệnh kiểm tra và commit đầy đủ được giữ trong log/traceability của repository.

Nguồn: backend/tests/evidence/vt024-traceability.md

```
# VT-024 Test Traceability

The repository tests retain their existing Python function names. This report assigns documentation IDs T01–T14 to the
  14 retry/backfill tests; these IDs do not rename tests. Commit hashes show where tests/evidence were introduced or
  corrected. API persistence regression and standalone experiments are outside T01–T14.

| Acceptance criterion / risk | Test Case ID | Test function / experiment | Test file | Commit SHA | Actual result |
|---|---|---|---|---|---|
| AC-01: transient HTTP 5xx retry | T06 | `test_http_500_is_retried_then_succeeds` |
  `backend/tests/test_rate_limit.py` | `11055810f10729963a8706b22fec2fcf5d9b0647` | Pass; first 500 then 200. |
| AC-01: Retry-After behavior | T01 | `test_rate_limit_retry_after_is_respected` | `backend/tests/test_rate_limit.py`
  | `11055810f10729963a8706b22fec2fcf5d9b0647` | Pass; configured delay is respected. |
| AC-01: timeout retry | T07 | `test_read_timeout_is_retried_then_succeeds` | `backend/tests/test_rate_limit.py` |
  `11055810f10729963a8706b22fec2fcf5d9b0647` | Pass; first ReadTimeout then 200. |
| AC-01: connection error retry | T08 | `test_connection_error_is_retried_then_succeeds` |
  `backend/tests/test_rate_limit.py` | `11055810f10729963a8706b22fec2fcf5d9b0647` | Pass; first ConnectError then 200.
  |
| AC-02: bounded attempts, success only after success, exhausted transport error propagation | T10 |
  `test_timeout_retry_is_bounded_and_returns_only_after_success` | `backend/tests/test_rate_limit.py` | Test added in
  `11055810f10729963a8706b22fec2fcf5d9b0647`; oracle corrected in `ecd22676f6a1ce43bcfa0d21852f717d475c130d` | Pass;
  40 passing Hypothesis examples, 2 invalid generated examples, 0 failures. It asserts at most `max_retries + 1` calls
  and propagates `httpx.TransportError` on exhaustion. |
| AC-02: non-retryable auth error | T03 | `test_bad_token_is_not_retried` | `backend/tests/test_rate_limit.py` |
  `11055810f10729963a8706b22fec2fcf5d9b0647` | Pass; one attempt, 401 error propagated. |
| AC-02: endpoint allowlist | T04 | `test_content_endpoint_is_blocked_before_network` |
  `backend/tests/test_rate_limit.py` | `11055810f10729963a8706b22fec2fcf5d9b0647` | Pass; blocked before network. |
| AC-02: pagination Link handling | T05 | `test_pagination_follows_link_header` | `backend/tests/test_rate_limit.py` |
  `11055810f10729963a8706b22fec2fcf5d9b0647` | Pass; follows next page. |
| AC-02: unsafe request not retried on transient HTTP status | T09 |
  `test_transient_http_error_on_post_is_not_retried` | `backend/tests/test_rate_limit.py` |
  `11055810f10729963a8706b22fec2fcf5d9b0647` | Pass; POST receives 500 once and raises. |
| AC-02: retry exhaustion does not return success | T02 | `test_rate_limit_exhaustion_raises` |
  `backend/tests/test_rate_limit.py` | `11055810f10729963a8706b22fec2fcf5d9b0647` | Pass; repeated 403 ends in
  `GitHubAPIError`. |
| AC-03: duplicate/reordered ingestion retains unique business records | T13 |
  `test_generated_duplicate_issue_ids_remain_unique_after_reordered_reingestion` | `backend/tests/test_backfill.py` |
  `11055810f10729963a8706b22fec2fcf5d9b0647` | Pass; 30 passing Hypothesis examples, 4 invalid generated examples, 0
  failures. |
| AC-03: repeated backfill is idempotent for business rows | T11 | `test_backfill_is_idempotent_and_metadata_only` |
  `backend/tests/test_backfill.py` | `7c13113429ffb279c0128bf8030d5804ad35933e` | Pass; business-row counts remain
  stable. RawEvent/audit counts after the second run are not asserted. |
| AC-04: service persistence failure never returns successful result; recovery works | T14 |
  `test_commit_failure_never_returns_success_and_recovery_is_consistent` | `backend/tests/test_backfill.py` | Test
  introduced in `11055810f10729963a8706b22fec2fcf5d9b0647`; event alias/cleanup review in
  `ecd22676f6a1ce43bcfa0d21852f717d475c130d` | Pass on correct implementation; raises the injected SQLAlchemyError,
  verifies no issue/PR rows after rollback, then recovers. It killed the deliberate false-success mutant. |
| AC-04: API persistence error not reported as success | Not assigned (outside T01–T14) |
  `test_sync_api_does_not_return_success_when_persistence_fails` | `backend/tests/test_tenants_sync_api.py` |
  `e4e56c86b9dab3e3e3c2fbaf36b751f29e4b135a` | Pass; authenticated request with offline fake client returns HTTP 500
  and repo remains pending. |
| AC-04: deliberate false-success mutation | T14 (mutation experiment) | Mutated `BackfillService.run()` commit block,
  then ran `test_commit_failure_never_returns_success_and_recovery_is_consistent` | `backend/tests/test_backfill.py`;
  output/diff in `backend/tests/evidence/vt024-mutation.txt` and `vt024-mutation-diff.patch` | Test in
  `11055810f10729963a8706b22fec2fcf5d9b0647`; mutation evidence in `e4e56c86b9dab3e3e3c2fbaf36b751f29e4b135a` | Mutant
  killed: test failed with `DID NOT RAISE SQLAlchemyError`, as expected. Mutant absent from submission branch. |
| AC-05: partial ingestion status/data behavior | T12 | `test_backfill_marks_partial_when_one_resource_fails` |
  `backend/tests/test_backfill.py` | `7c13113429ffb279c0128bf8030d5804ad35933e` | Pass; review endpoint failure
  produces `partial`, records an error, and successful issue/PR data remain. |
| Performance: 100 concurrent mocked tasks and p95 sample | Not assigned (standalone harness) | `main` / `_run_task` |
  `backend/tests/perf_retry_harness.py` | Initial harness in `e4e56c86b9dab3e3e3c2fbaf36b751f29e4b135a`; 100-worker
  barrier and corrected reporting in `5f1fcbb719a8efad847e58bca03dce406566bfae` | Pass: all 100 workers rendezvous
  before timing; 95 succeeded, 5 intentionally exhausted; 115 requests; 5% error rate; latest mock sample p50 0.376
  ms, p95 0.701 ms, max 1.131 ms. Not production/API/database performance. |

## Commit chain

- Baseline: `90f831c8c8afc84dd4f0268b7ed5704e41f32cbf`
- Existing backfill tests: `7c13113429ffb279c0128bf8030d5804ad35933e`
- RED test commit: `11055810f10729963a8706b22fec2fcf5d9b0647`
- GREEN production retry and oracle correction: `ecd22676f6a1ce43bcfa0d21852f717d475c130d`
- Phase 4 API/coverage/performance evidence commit: `e4e56c86b9dab3e3e3c2fbaf36b751f29e4b135a`
- Checkpoint 6 performance refinement and corrected experiment: `5f1fcbb719a8efad847e58bca03dce406566bfae`
```

Nguồn: backend/tests/evidence/vt024-phase4-context.txt

```
VT-024 Phase 4 evidence context

Repository / revision
- Branch: bt3-23001819-vt024-retry
- GREEN implementation commit: ecd22676f6a1ce43bcfa0d21852f717d475c130d
- Preserved RED test-only commit: 11055810f10729963a8706b22fec2fcf5d9b0647
- Phase 4 evidence was collected against the GREEN commit plus the uncommitted Phase 4
  test/harness/dependency/evidence changes.
- No GitHub API or database was used by the performance harness.
- The original RED evidence files remain untouched.

Environment
- Python 3.13.5
- pytest 8.4.2
- Hypothesis 6.170.15
- httpx 0.28.1
- SQLModel 0.0.48
- FastAPI 0.162.0
- Ruff 0.17.3
- coverage.py 7.17.3 (installed after user approval; added to the optional dev dependency set)
- mypy is unavailable (`python -m mypy app` reports `No module named mypy`).

Test quality and regression checks
- API-level sync persistence test: 1 passed. It authenticated as tenant owner, used a local fake GitHub client and an
  injected SQLAlchemy commit failure, observed HTTP 500, and verified the repository stayed pending.
- Retry/backfill/API tests in two different file orders: 15 passed in each run; output and commands are in
  vt024-flaky-orders.txt.
- Full backend suite after final Phase 4 changes: 49 passed; output is in vt024-full-suite.txt.
- Ruff on production retry client and relevant tests/harness: passed; output is in vt024-ruff.txt.
- mypy was attempted and unavailable in this environment; no type check result is claimed.
- Persistence-failure regression is included in the full suite and killed the deliberate false-success mutant.

Property-based checks
- Retry property: 40 passing examples, 2 invalid, 0 failing or minimized counterexamples.
- Backfill idempotency property: 30 passing examples, 4 invalid, 0 failing or minimized counterexamples.
- Both are deterministic (`derandomize=True`, `database=None`) and passed when rerun together.
- Neither property uses `assume()` or `.filter()`. A no-op probe with the same bounded strategies also reported
  invalid generated examples, so these were generated-data invalids rather than application-level rejected inputs.
  Hypothesis output did not expose byte-level causes for individual invalid examples; the exact individual examples
  therefore cannot be identified from the available statistics.
- Retry assertions still enforce calls <= max_retries + 1, success only after a response, and propagation of transport
  exceptions on exhaustion. The oracle accepts the existing `httpx.TransportError` family rather than imposing
  GitHubAPIError conversion.

Mutation experiment
- A temporary local branch was created from the GREEN commit. The experiment changed only the BackfillService
  persistence block to catch a commit exception and return a fabricated `ready` result.
- Command: `python -m pytest -q
  tests/test_backfill.py::test_commit_failure_never_returns_success_and_recovery_is_consistent`
- Actual result: failed as intended with `Failed: DID NOT RAISE <class 'sqlalchemy.exc.SQLAlchemyError'>`; this
  demonstrates that the regression test detects the false-success defect.
- Exact experiment output and unified mutant diff are in vt024-mutation.txt and vt024-mutation-diff.patch.
- The mutated source was restored before continuing on the assignment branch. No mutant was committed; the temporary
  local branch reference is removed after final verification.

Coverage
- Command: `python -m coverage run --branch --source=app.github.client,app.ingestion.backfill -m pytest -q
  tests/test_rate_limit.py tests/test_backfill.py tests/test_tenants_sync_api.py`
- Result: 15 passed.
- Command: `python -m coverage report -m`
- The text report's 74%/92%/83% `Cover` values combine statements and branch arcs. Exact statement coverage is
  client.py 95/121 = 78.51%, backfill.py 151/163 = 92.64%, total 246/284 = 86.62%. Exact branch coverage is 31/50 =
  62.00%, 35/40 = 87.50%, total 66/90 = 73.33%. Selected modules only; full machine-readable details are in
  vt024-coverage.json and interpretation/gaps in vt024-coverage-review.md.
- This is scoped line and branch coverage for the selected vertical slice, not whole-repository coverage.

Concurrent mocked performance harness
- Harness: tests/perf_retry_harness.py; 100 tasks/workers, each owns a GitHubClient, MockTransport, and attempt
  counter. No network/database and no shared mutable state.
- Retry configuration: max_retries=2; 90 immediate-success tasks, 5 transient-then-success tasks, and 5 intentionally
  exhausted 503 tasks.
- Actual observed aggregate: 95 successes, 5 exhausted; 115 attempts total (90×1 + 5×2 + 5×3).
- All 100 worker tasks wait on a `threading.Barrier` before their timed request work. The barrier wait is excluded
  from per-task latency. Handlers perform no blocking operation; GitHub transport is `httpx.MockTransport`, and the
  retry sleep callback is a no-op.
- Latency covers the request and retry code path for every task, including exhaustion. It includes retry/backoff
  calculation and calls to the no-op sleep callback, but it excludes real backoff waiting. Nearest-rank p50/p95/max in
  the rerun were 0.376/0.701/1.131 ms.
- This experiment does not establish production p95, real GitHub network latency, database throughput, or full API
  concurrency. The 3.0 s requirement is only compared with this isolated mock scenario, not validated for production.
- Exact output is in vt024-performance.txt.

Warnings and limitations
- Pytest reports the existing Starlette deprecation warning that its TestClient currently uses httpx; it does not fail
  the suite.
- Coverage reflects only `app.github.client` and `app.ingestion.backfill` under the selected retry/backfill/API tests.
- PoolTimeout is a TimeoutException and is retried for GET, adding bounded delay if local pool capacity is the cause.
```

```
WriteTimeout is also covered; retry is still GET-only. Existing GET 403 handling remains intact, so a permanent
permission 403 can incur bounded delay before failing. Unsafe methods do not retry transport or 403/429/5xx
failures.
```